



**WYDZIAŁ
ELEKTROTECHNIKI
I INFORMATYKI**
POLITECHNIKI RZESZOWSKIEJ

Jakub Skrzynecki

Predykcja incydentów przy użyciu AI w środowisku SOC

PROJEKT

Rzeszów, 2026

Spis treści

1. Wstęp i cel projektu	4
2. Wymagania i założenia	5
2.1. Wymagania funkcjonalne	5
2.2. Wymagania niefunkcjonalne	6
3. Środowisko projektowe i technologie	8
3.1. Stos technologiczny	8
3.2. Struktura repozytorium	9
4. Architektura systemu	11
4.1. Diagram sekwencji jednego cyklu	12
5. Fazy realizacji projektu	14
5.1. Faza 1: Konfiguracja środowiska i weryfikacja SSH	14
5.2. Faza 2: Rdzeń agenta SOC	15
5.3. Faza 3: Warstwa analizy AI i reakcji	15
5.4. Faza 4: Finalne PoC i testy sekwencyjne	16
6. Szczegółowy opis komponentów	17
6.1. soc_agent.py: orkiestracja i polityka reakcji	17
6.2. ai_analyzer.py: klient Ollama i kontrakt JSON	18
6.3. fim_monitor.py: monitor integralności plików	19
6.4. alert_system.py i reporting_engine.py	19
7. Działanie systemu krok po kroku	20
7.1. Reakcja systemu na wynik analizy	20
8. Wyniki testów na realnych atakach	23
9. Wnioski i podsumowanie	29
9.1. Wnioski techniczne	29
9.2. Wnioski architektoniczne	29
9.3. Analiza przyczyn niedoszacowania zagrożeń	30
9.4. Walidacja wprowadzonych poprawek (iteracje v2 i v2c)	31
9.5. Dalsze kierunki rozwoju	34

1. Wstęp i cel projektu

Rozwój technologii sztucznej inteligencji otwiera nowe możliwości w dziedzinie cyberbezpieczeństwa, w tym automatyzacji zadań analitycznych wykonywanych dotychczas przez ludzi. Centra operacji bezpieczeństwa (Security Operations Center, SOC), zgodnie z definicją przedstawioną w wytycznych NIST [1], przetwarzają każdego dnia ogromne ilości logów i alertów, których właściwa archiwizacja i przegląd opisane są w uzupełniającym przewodniku NIST SP 800-92 [2], co w praktyce prowadzi do zjawiska określanego jako alert fatigue, czyli przeciążenia analityków nadmiarową liczbą zdarzeń. Zastosowanie lokalnych modeli językowych (LLM) jako warstwy wstępnej analizy może odciążyć specjalistów od rutynowych czynności klasyfikacji incydentów, pozostawiając im czas na działania wymagające wiedzy eksperckiej [10, 11].

Projekt to Proof of Concept (PoC) systemu SOC wspomaganego przez lokalne AI. Cel główny zdefiniowano następująco:

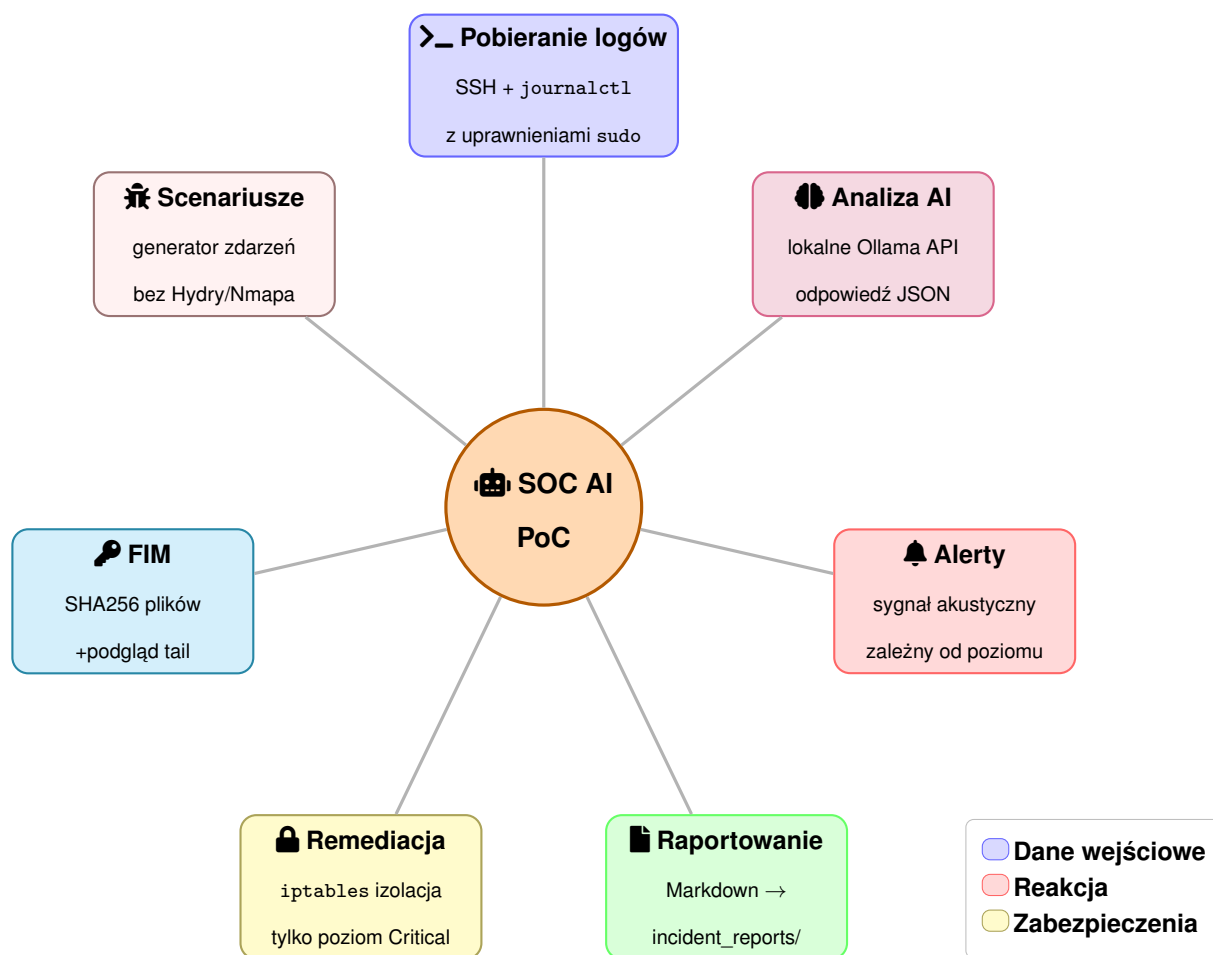
Zaprojektowanie i implementacja bezagentowego systemu monitorowania bezpieczeństwa maszyny Linux z poziomu hosta Windows, w którym lokalny model językowy (Ollama/Llama) pełni rolę automatycznego analityka klasyfikującego incydenty i proponującego remediację.

Zakres realizacyjny obejmuje cztery obszary: budowę agenta SOC pobierającego logi przez SSH bez instalacji oprogramowania na VM, integrację z lokalnym modelem Ollama zwracającym decyzję w JSON, moduł monitorowania integralności plików (FIM) [7] wykrywający nieautoryzowane zmiany konfiguracyjne oraz zautomatyzowane reakcje (alerty dźwiękowe, raporty Markdown, izolacja sieciowa dla zagrożeń krytycznych). Całość zweryfikowano sześcioma realnymi atakami na VM, opisanymi w rozdziale 8.

2. Wymagania i założenia

2.1. Wymagania funkcjonalne

Wymagania funkcjonalne sformułowane na etapie projektowania systemu zilustrowano radialnie na rysunku 2.1. Każda z siedmiu gałęzi odpowiada jednemu z podsystemów agenta i jest bezpośrednim przełożeniem celu głównego pracy na zakres implementacyjny. Pogrupowanie kolorystyczne (dane wejściowe, reakcja, zabezpieczenia) wyznacza trzy warstwy logiczne, w których działają komponenty agenta, a linie zygzakowate symbolizują niejednorodność interfejsów między orkiestratorem a poszczególnymi modułami: pozyskiwanie danych odbywa się przez SSH i system plików, reakcja przez winsound i system plików, a remediacja przez iptables.



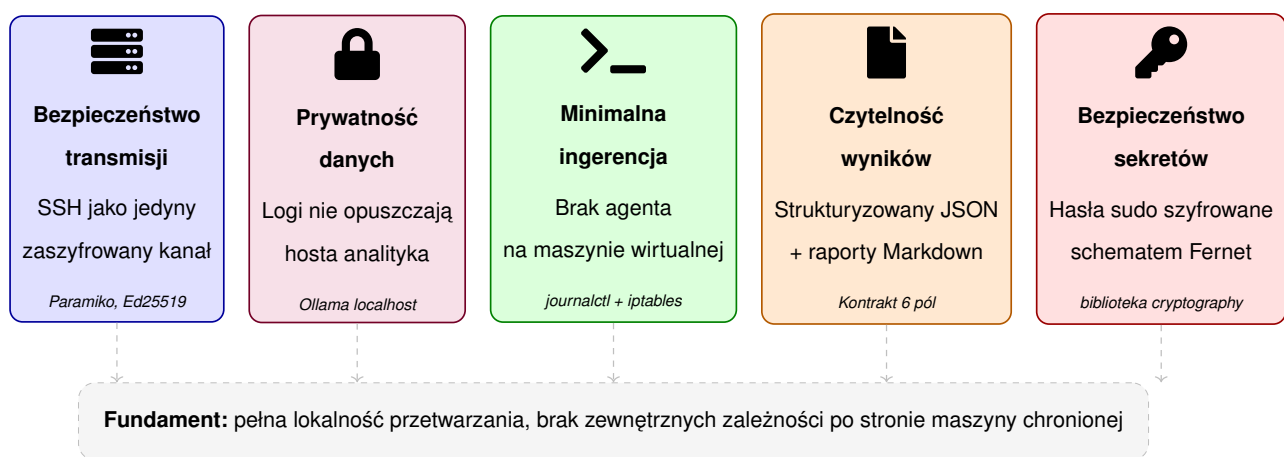
Rys. 2.1. Siedem wymagań funkcjonalnych systemu SOC (PoC) zorganizowanych radialnie wokół centralnego komponentu. Każda gałąź wymagania zawiera kluczową technologię realizującą daną funkcję.

Projekt obejmuje pełną pętlę reagowania na incydent: pobieranie zdarzeń, analizę przez lokalny model językowy, powiadomienie oraz utrwalenie raportu, a także opcjonalną remediację realizowaną wyłącznie dla zagrożeń krytycznych. Założono, że kanał pozyskiwania danych opiera

się na pojedynczym połączeniu SSH i na poleceniu `journalctl` wykonywanym z uprawnieniami `sudo`, dzięki czemu po stronie maszyny monitorowanej nie trzeba instalować żadnego dedykowanego komponentu. Wymaganie dotyczące remediacji ograniczono do automatycznej izolacji sieciowej maszyny w przypadku zagrożeń o poziomie Critical, zgodnie z zasadą najmniejszego skutku ubocznego sformułowaną w wytycznych NIST SP 800-61 [1]. Komponent FIM uzupełnia warstwę logów o deterministyczną kontrolę integralności krytycznych plików konfiguracyjnych, dzięki czemu warstwa AI otrzymuje dwa rozłączne strumienie wejściowe: behawioralny oraz strukturalny. Ostatnie wymaganie, dotyczące generowania zdarzeń testowych z poziomu hosta, pozwoliło uniknąć użycia ofensywnych narzędzi pokroju Hydra czy Nmap i zachować pełną powtarzalność scenariuszy walidacyjnych.

2.2. Wymagania niefunkcjonalne

Wymagania niefunkcjonalne definiują jakościowe ramy realizacji systemu i jego osadzenie w kontekście pracy analityka SOC. Pięć obszarów uznanych za krytyczne dla rozwiązania PoC operującego na rzeczywistych logach uwierzytelnienia (bezpieczeństwo transmisji, prywatność przetwarzanych danych, minimalną ingerencję w maszynę docelową, czytelność produktów analizy oraz bezpieczeństwo sekretów) zilustrowano na rysunku 2.2 jako pięć filarów wspartych na wspólnym fundamencie lokalności przetwarzania. Każdy filar opatrzono technologią lub mechanizmem, który realizuje dane wymaganie w bieżącej iteracji projektu.



Rys. 2.2. Pięć obszarów wymagań niefunkcjonalnych systemu SOC AI PoC jako filary konstrukcyjne nad wspólnym fundamentem lokalności przetwarzania. Każdy filar zawiera kluczową technologię realizującą daną zasadę.

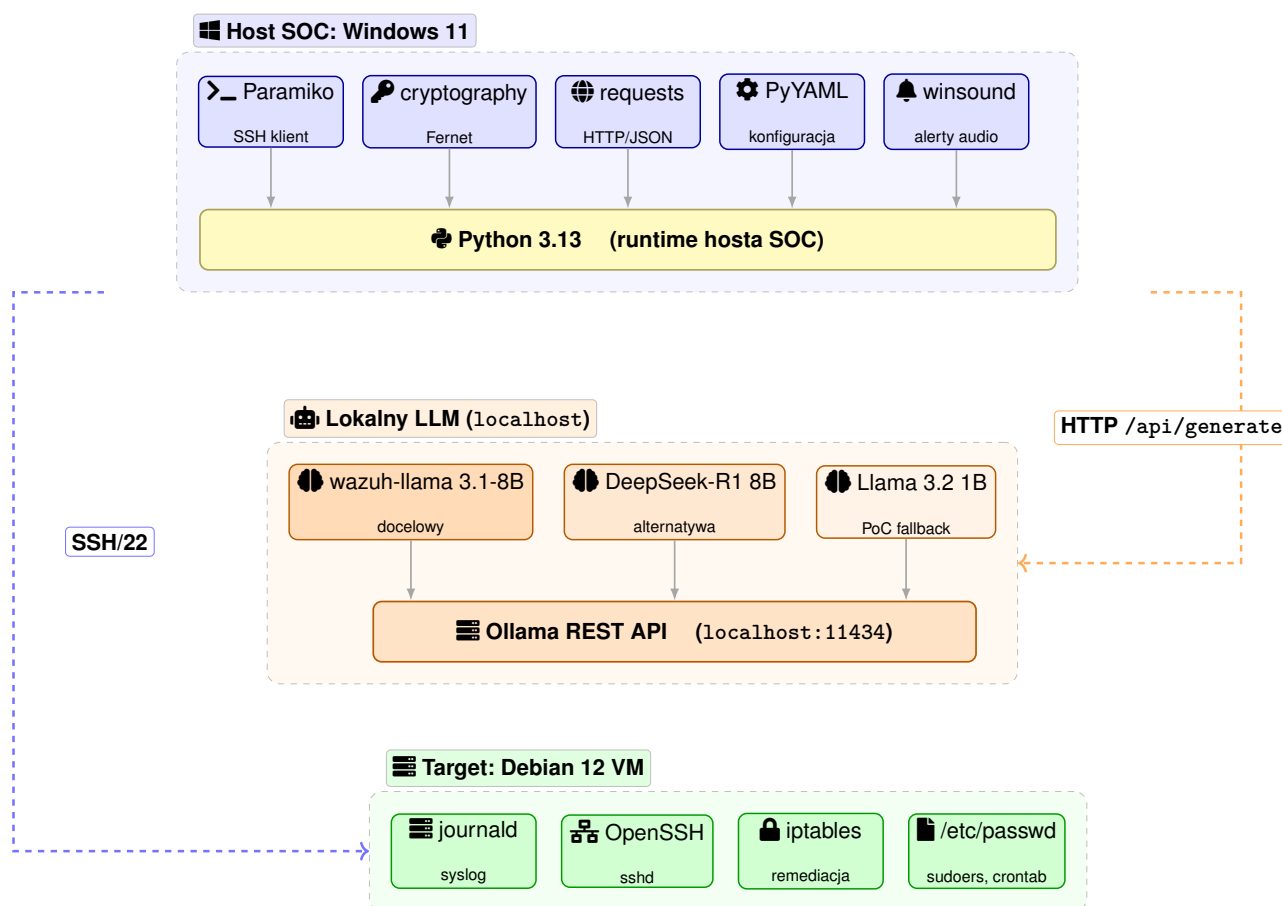
Wymóg pojedynczego zaszyfrowanego kanału komunikacji wykluczył protokoły niezaszyfrowane (Telnet, nieautentykowane interfejsy `syslog`) i przesądził o wyborze SSH realizowanego przez bibliotekę Paramiko [4]. Zasada prywatności danych przesądziła o uruchomieniu silnika

językowego lokalnie, w ramach serwera Ollama [5] pod adresem localhost, co eliminuje ryzyko wycieku zawartości logów do zewnętrznych dostawców usług AI. Wymóg minimalnej ingerencji w VM utrwalił bezagentową filozofię projektu: cały kod aplikacyjny działa po stronie hosta analityka. Założenie czytelności wyników skutkowało wymuszeniem na modelu odpowiedzi w formacie JSON o ustalonym schemacie, co umożliwiło deterministyczne parsowanie i uproszczone budowanie raportów Markdown. Ochronę sekretów zrealizowano przez moduł SecretsManager z symetrycznym schematem Fernet z biblioteki cryptography [8], dzięki czemu hasło konta administracyjnego nie pozostaje w pamięci procesu w formie jawnej dłużej niż to konieczne.

3. Środowisko projektowe i technologie

3.1. Stos technologiczny

Stos technologiczny przedstawia rysunek 3.1 w formie trzech stref odzwierciedlających trójwarstwową architekturę host-gość-AI. Dobór elementów wynikał z trzech kryteriów: pełnej lokalności przetwarzania, kompatybilności z heterogenicznym środowiskiem host-gość oraz minimalizacji zewnętrznych zależności po stronie VM. Po stronie hosta Python 3.13 stanowi bazę aplikacyjną z warstwą bibliotek (Paramiko, cryptography, requests, PyYAML, winsound). Strefa Ollama uruchamia wymiennie trzy modele LLM [13, 14, 12], a strefa VM wykorzystuje wyłącznie systemowe komponenty Debiana 12 (journald, OpenSSH, iptables, krytyczne pliki konfiguracyjne).



Rys. 3.1. Stos technologiczny SOC AI PoC w podziale na trzy strefy: host Windows (orkiestrator + biblioteki), lokalny LLM (Ollama z trzema modelami) oraz target Debian (komponenty systemowe). Strzałki pokazują dwa kanały komunikacji: SSH (host → VM) i HTTP (host → Ollama).

Po stronie języka aplikacyjnego główną rolę odgrywa Paramiko [4], zapewniający pełną kontrolę nad pseudoterminalem i wywołania sudo -S bez modyfikacji sudoers, oraz cryptography z implementacją Fernet [8] jako podstawa zaszyfrowanej pamięci sekretów.

Warstwa AI została zaprojektowana tak, aby umożliwiać wymianę modelu bez zmian w kodzie agenta. Modelem docelowym jest OpenNix/wazuh-llama-3.1-8B-v1 [13] (Llama 3.1 8B dostrojona na logach Wazuh i typowych zdarzeniach SIEM), alternatywą deepseek-r1:8b [14] bazujący na innej architekturze treningowej. Pierwsza runda PoC korzystała ze skwantyzowanego modelu Llama 3.2 1B [12] ze względu na ograniczenia RAM, zastąpionego docelowym 8B w rundzie walidacji realnych ataków (sekcja 8). Po stronie VM system korzysta wyłącznie z komponentów systemowych: journald (syslog RFC 5424 [9]) i iptables [6] dla izolacji sieciowej. Po stronie hosta alerty akustyczne realizuje winsound z biblioteki standardowej, bez zewnętrznych pakietów multimedialnych.

3.2. Struktura repozytorium

Kod źródłowy zorganizowano w układzie odzwierciedlającym kolejne fazy realizacji opisane w rozdziale 5. Granice między fazami pokrywają się z granicami modułów Pythona, dzięki czemu każdy etap można walidować w izolacji. Katalogi runtime (logs_from_virtual_machine/, incident_reports/) powstają dopiero podczas pracy agenta i dostarczają materiału dowodowego. Rysunek 3.2 przedstawia drzewo katalogów wraz z przypisaniem klas implementujących funkcjonalność.



katalog gtowny projektu

faza 1: setup srodowiska

weryfikacja polaczenia SSH (Paramiko)

faza 2: rdzen agenta

klasa SSHLogCollector (pobieranie logow)

klasa FIMMonitor (SHA256 plikow)

klasa SecretsManager (szyfrowanie Fernet)

szyfrowanie poswiadczen z machine_info.md

faza 3: warstwa AI i reakcja

klient Ollama, kontrakt JSON

sygnal akustyczny (winsound, syrena)

raporty Markdown w incident_reports/

faza 4: integracja i testy

jednokrotne wywołanie run_cycle()

6 funkcji simulate_ (zdarzenia testowe)*

GLOWNY orkiestrator klasy SOCAgent

konfiguracja agenta (interwały, IP)

katalog runtime na artefakty agenta

surowe logi pobrane z VM

slady AI (input + raw + normalized JSON)

baseline SHA256 monitorowanych plikow

raporty Markdown Low/Medium/High/Critical

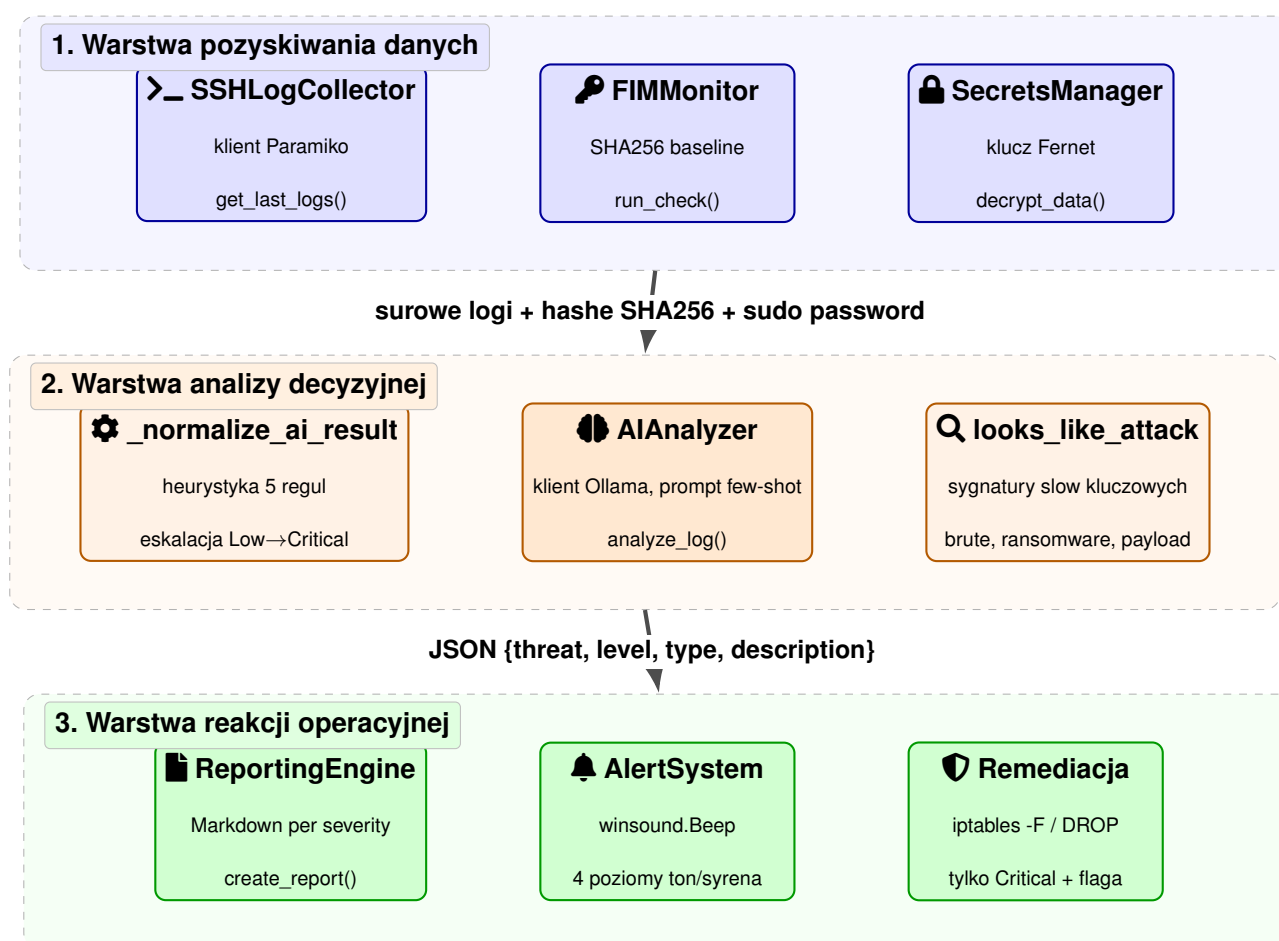
! *poza VCS (.gitignore)*

Rys. 3.2. Drzewo katalogów repozytorium SOC AI PoC. Cztery niebieskie gałęzie odpowiadają fazom realizacji projektu (Phase 1–4), katalogi pomarańczowe powstają w trakcie działania agenta (runtime), a zielone i czerwone wyróżniają odpowiednio konfigurację oraz sekrety wykluczone z kontroli wersji.

Pliki credentials.enc, secret.key i fim_state.json pozostają poza kontrolą wersji, zgodnie z higieną zarządzania sekretami z NIST SP 800-92 [2].

4. Architektura systemu

Architektura logiczna agenta dzieli przetwarzanie na trzy warstwy funkcjonalne: pozyskiwanie danych, analiza decyzyjna oraz reakcja operacyjna. Każda warstwa odpowiada za inny etap potoku przetwarzania zdarzeń bezpieczeństwa, a granice między nimi pokrywają się z granicami klas Python, co umożliwia niezależne testowanie. Rysunek 4.1 przedstawia ten potok jako trzy poziome paski, w których strzałki wskazują kierunek przepływu informacji: surowe logi i hashe SHA256 z VM, ustrukturyzowana decyzja JSON od modelu językowego oraz akcje wyjściowe (alert dźwiękowy, raport Markdown, polecenia iptables).



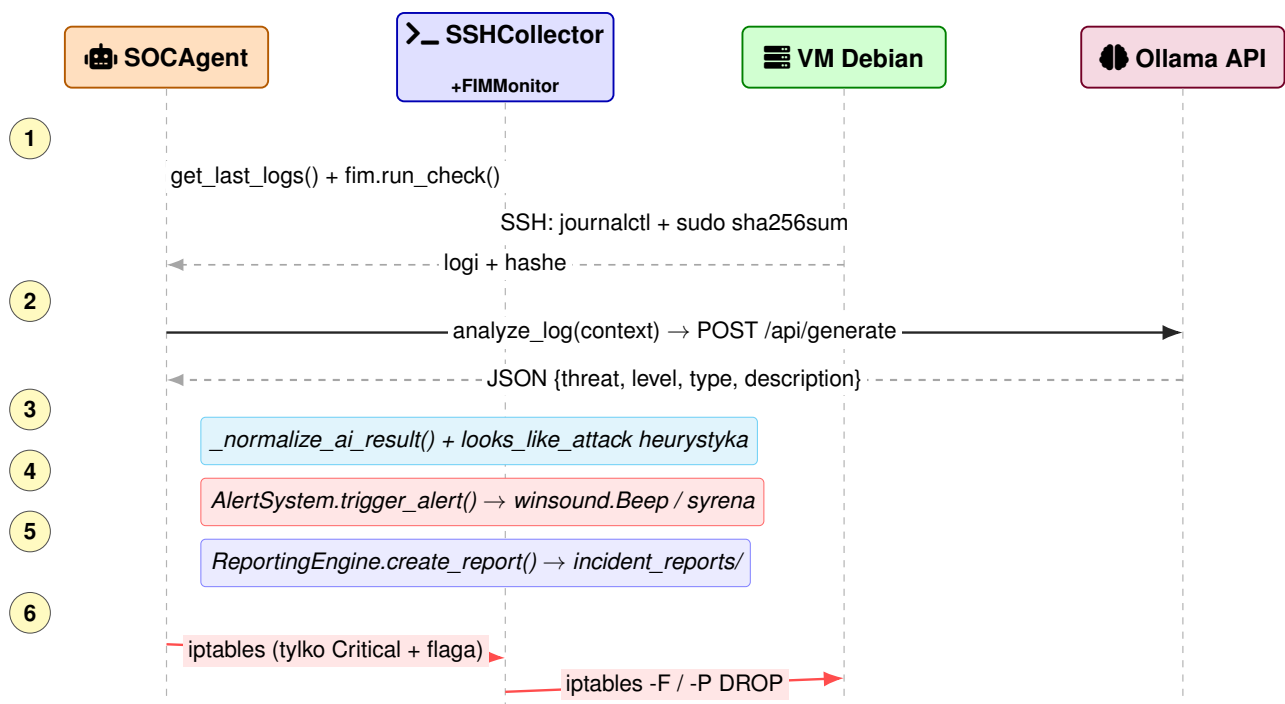
Rys. 4.1. Architektura logiczna agenta SOC w podziale na trzy warstwy funkcjonalne. Strzałki pokazują przepływ danych: logi i hashe trafiają z warstwy pozyskiwania do analizatora, ustrukturyzowana decyzja JSON aktywuje odpowiednie kanały reakcji.

Diagram wyróżnia trzy poziomy odpowiedzialności. Warstwa pozyskiwania danych zbiera materiał dowodowy z VM przez pojedynczy kanał SSH: SSHLogCollector pobiera logi uwierzytelniania poleceniem `journalctl`, FIMMonitor weryfikuje integralność krytycznych plików konfiguracyjnych przez `sha256sum`, a SecretsManager dostarcza poświadczeń `sudo` z zaszy-

frowanego pliku Fernet. Warstwa analizy decyzyjnej ma trzy etapy w sekwencji: AIAntyzer wysyła kontekst do lokalnego modelu Ollama i otrzymuje strukturyzowaną odpowiedź JSON, heurystyka looks_like_attack przeszukuje opis i powiązane logi pod kątem sygnatur ataku, a procedura _normalize_ai_result koryguje typowe błędy modelu (false negative, niedoszacowanie poziomu). Warstwa reakcji operacyjnej kanalizuje wynik na trzy niezależne wyjścia: alert dźwiękowy AlertSystem dla operatora, trwały raport Markdown w katalogu incident_reports oraz, wyłącznie dla zagrożeń Critical z aktywną flagą bezpiecznika, sieciową izolację maszyny poleceniami iptables.

4.1. Diagram sekwencji jednego cyklu

Rysunek 4.2 przedstawia w notacji UML sześć głównych operacji wykonywanych przez agenta w pojedynczym przejściu metody run_cycle (powtarzane co poll_interval_s sekund). Cztery linie życia odpowiadają uczestnikom wymiany: orkiestrator SOCAgent, kolektor SSH wraz z modulem FIM, monitorowana VM Debian oraz lokalny serwer Ollama.



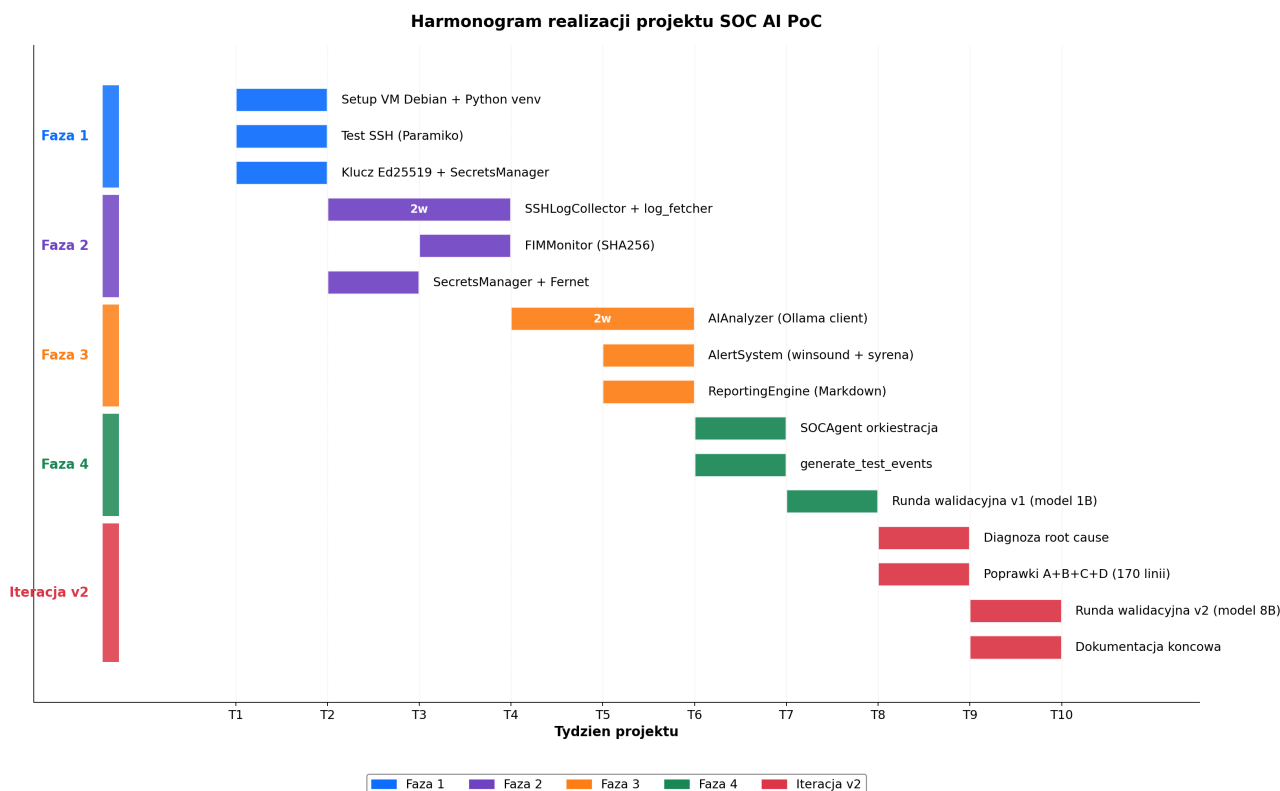
Rys. 4.2. Diagram sekwencji UML jednego cyklu run_cycle() agenta SOC. Sześć ponumerowanych kroków: pozyskanie logów + FIM, analiza AI, normalizacja, alert dźwiękowy, raport Markdown oraz remediacja sieciowa. Strzałki czerwone (krok 6) oznaczają ścieżkę krytyczną aktywną tylko dla zagrożeń Critical z ustawioną flagą bezpiecznika.

Przepływ jest deterministyczny i liniowy do otrzymania odpowiedzi modelu. Krok 1 realizuje pozyskiwanie danych przez pojedynczy kanał SSH (journalctl dla logów uwierzytelniania oraz sha256sum dla integralności plików). Krok 2 przekazuje kontrolę warstwie AI

z wymuszonym formatem odpowiedzi JSON. Krok 3 wykonuje lokalną normalizację wyniku przez heurystykę `looks_like_attack`, korygując typowe błędy modelu. Kroki 4 i 5 materializują reakcję informacyjną (alert dźwiękowy i raport Markdown). Krok 6 to remediacja sieciowa iptables [6], aktywowana wyłącznie dla zagrożeń Critical z jawnie ustawioną flagą bezpiecznika, co zapobiega przypadkowej utracie dostępu do maszyny chronionej.

5. Fazy realizacji projektu

Projekt zrealizowano w czterech sekwencyjnych fazach inżynierskich, uzupełnionych dwiema rundami walidacyjnymi (rozdział 9). Harmonogram przedstawia rysunek 5.1. Iteracja v2 (zaznaczona na czerwono) wprowadzona po empirycznej walidacji v1 objęła wyłącznie warstwy decyzyjne (prompt, heurystyka, progi remediacji), bez modyfikacji warstw transportowej ani analitycznej.



Rys. 5.1. Harmonogram realizacji projektu SOC AI PoC w podziale na cztery fazy implementacyjne i iterację v2 (diagnoza root cause + naprawa). Każda faza grupuje zadania rozwijane równolegle; strzałki adnotacji wskazują kluczowe punkty walidacji.

5.1. Faza 1: Konfiguracja środowiska i weryfikacja SSH

Pierwsza faza obejmowała konfigurację środowiska: Python 3.13 z izolowanym .venv po stronie hosta Windows oraz Debiana 12 w hiperwizorze VMware jako monitorowany punkt końcowy z dziennikiem systemd-journal [9]. Głównym artefaktem był skrypt test_ssh_v1.py oparty na bibliotece paramiko [4], który weryfikował połączenie SSH oraz wywołania sudo -S whoami z hasłem przekazywanym na strumień wejściowy. Równolegle wygenerowano parę kluczy Ed25519 (część publiczna dopisana do authorized_keys konta it-admin) oraz przygo-

towano `encrypt_machine_info.py` szyfrujący poświadczenia do `credentials.enc` przez moduł `SecretsManager`, zgodnie z zasadami higieny opisanymi w NIST SP 800-92 [2].

5.2. Faza 2: Rdzeń agenta SOC

Faza druga obejmowała implementację rdzenia: `SSHLogCollector` w module `log_fetcher.py` opakowuje klienta paramiko [4] i deleguje wywołania uprzywilejowane do metody `__exec_ptty_with_sudo`, która automatycznie przepisuje `sudo` na wariant `sudo -S` i wprowadza hasło na pseudoterminal. Główna metoda `get_last_logs` adaptacyjnie wybiera źródło logów (`/var/log/auth.log` lub `journalctl -t sshd -t audit`), deduplikuje wpisy i oczyszcza je z sekwencji ANSI. Zarządzanie sekretami wydzielono do klasy `SecretsManager` opartej na schemacie Fernet z biblioteki `cryptography` [8]. Trzecim filarem jest `FIMMonitor` z modułu `fim_monitor.py` utrzymujący baseline SHA256 plików `/etc/passwd`, `/etc/shadow`, `/etc/crontab` i `/etc/sudoers` w `fim_state.json`; metoda `run_check` porównuje aktualne hashe z baseline i w razie rozbieżności pobiera końcowe trzydzieści linii zmodyfikowanego pliku przez `sudo tail`.

5.3. Faza 3: Warstwa analizy AI i reakcji

W fazie trzeciej zaimplementowano warstwę decyzyjną agenta. Moduł `ai_analyzer.py` udostępnia klasę `AIAnalyzer` działającą jako klient Ollamy [5], która wysyła żądanie `POST /api/generate` pod `http://localhost:11434` z parametrem `format: "json"`, temperaturą `0,1` i limitem `num_predict=256`, co utrzymuje odpowiedzi deterministyczne i zwarte. Prompt systemowy wymusza schemat z polami `threat`, `level`, `type`, `description`, `remediation_plan` oraz `technical_action`, gdzie poziom ważności mapowany jest na czterostopniową skalę zgodną z NIST SP 800-61 [1]. Metoda `analyze_log` implementuje ponawianie z liniowo rosnącym backoffem; niepowodzenie zwraca `None`, traktowane przez orkiestrator jako pomijany cykl.

Moduł `alert_system.py` z klasą `AlertSystem` odpowiada za sygnalizację dźwiękową przez `winsound.Beep`: słownik `severity_tones` mapuje cztery poziomy na pary częstotliwość-czas trwania, a dla `Critical` generuje dwukrotny sweep w paśmie 800–2500 Hz z krokiem 200 Hz, poprzedzony tonem pre-roll. Moduł `reporting_engine.py` z klasą `ReportingEngine` tworzy raporty Markdown w `incident_reports/{severity}/`, zawierające opis, playbook, ostatnie pięć linii logów oraz znormalizowaną listę zmian FIM serializowaną przez `json.dumps`, zgodnie z rekomendacjami [1].

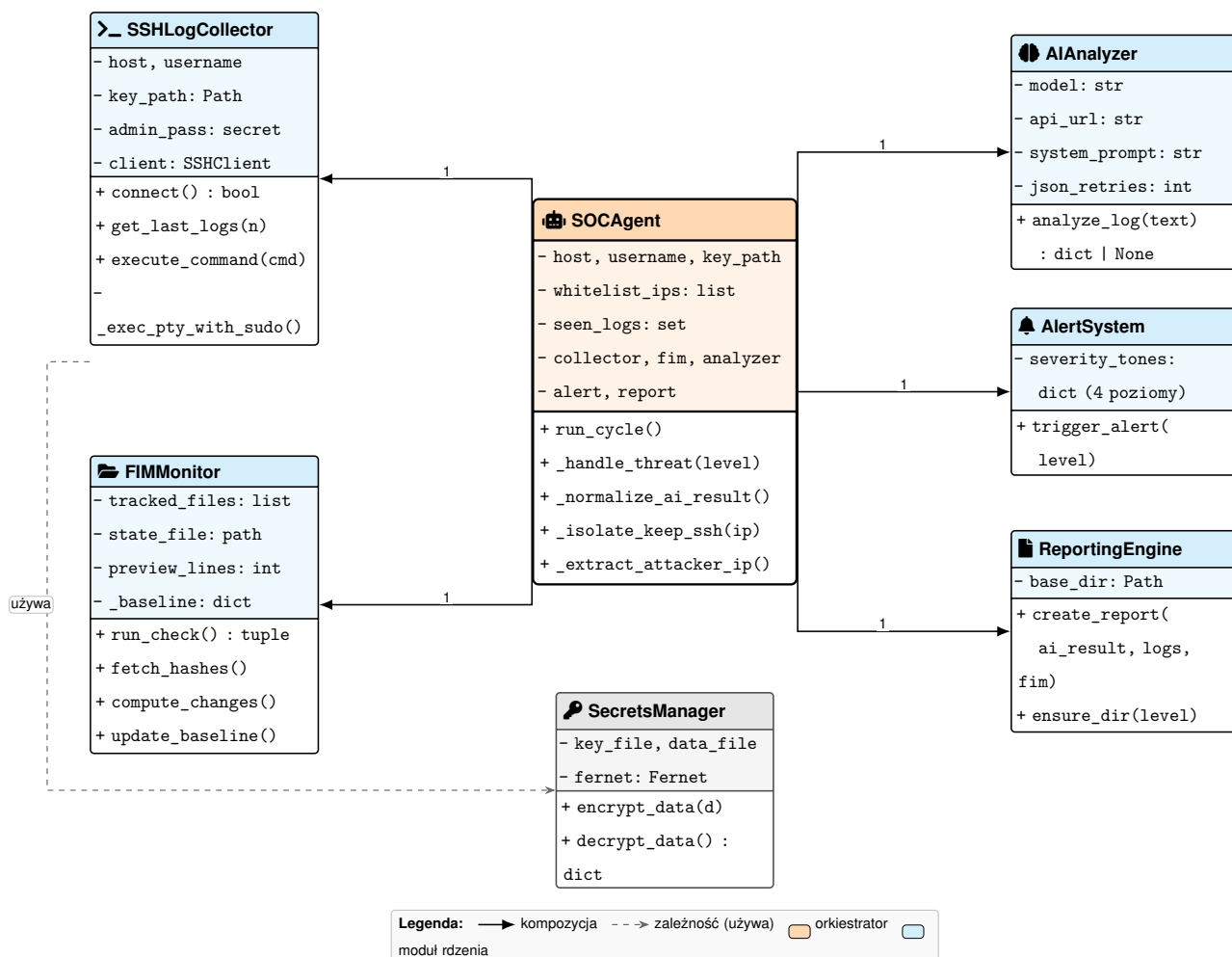
5.4. Faza 4: Finalne PoC i testy sekwencyjne

W fazie czwartej komponenty zintegrowano w klasie SOCAgent z soc_agent.py, która instancjonuje wszystkie moduły niższych warstw i wczytuje soc_config.yaml (interwał poll_interval_s, whitelist_ips, parametry Ollamy). Metoda run_cycle wykonuje pełen cykl: FIM, pobranie logów, deduplikacja seen_logs, wyselekcjonowanie linii podejrzanych wyrażeniem suspicious_re i wysłanie do AI. Po odpowiedzi modelu _normalize_ai_result koryguje typowe błędy (wymuszenie threat=true i Medium przy atakopodobnym opisie, usunięcie destrukcyjnych komend poniżej Critical), a ślad zapisywany jest jako JSON w ai_traces.

Polityka reakcji ma trzy poziomy. Dla High klasyfikowanych jako brute-force agent ekstrahuje adres atakującego wzorcem from <IPv4> i dodaje regułę iptables -I INPUT -s <ip> -j DROP [6]. Dla Critical aktywowana jest _isolate_keep_ssh, która resetuje tablice filtrów, ustawia polityki DROP i przywraca tylko ruch zwrotny oraz dostęp SSH hosta; procedura wymaga jawnej zmiennej SOC_ENABLE_CRITICAL_ISOLATION=1 jako bezpiecznika. Materiał testowy generuje skrypt generate_test_events.py wstrzykujący wpisy przez logger -t sshd/audit, dzięki czemu nie wymaga zewnętrznych narzędzi ofensywnych pokroju Hydra czy Nmap. Funkcje simulate_* odwzorowują techniki MITRE ATT&CK [3]: T1110 (brute-force), T1053.003 (cron persistence), T1486 (ransomware), T1056 (keylogger), stanowiąc podstawę pomiaru skuteczności klasyfikacji omawianego w kolejnej części pracy, w tym porównania z rozwiązaniami referencyjnymi jak Wazuh [7].

6. Szczegółowy opis komponentów

Wzajemne zależności klas i kierunki kompozycji pokazano na rysunku 6.1. Centralną pozycję zajmuje orkiestrator SOCAgent skompozowany 1:1 z wszystkimi modułami: SSHLogCollector i FIMMonitor (pozyskiwanie danych), AIAalyzer, AlertSystem, ReportingEngine (interpretacja i reakcja); SecretsManager dołączony przez relację uses.



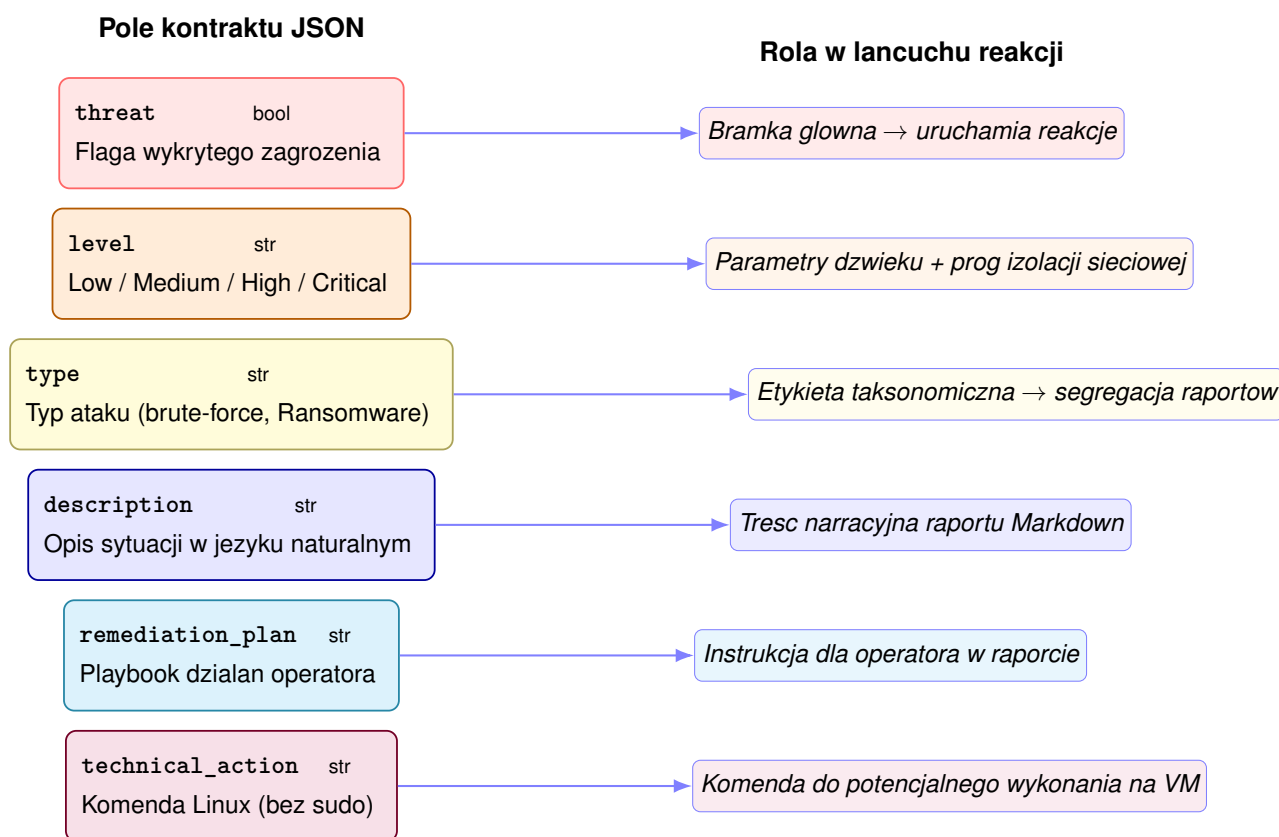
Rys. 6.1. Diagram klas UML kluczowych modułów agenta SOC: orkiestrator SOCAgent skompozowany z pięciu klas funkcjonalnych oraz pomocniczego SecretsManager (relacja uses). Każda klasa pokazana z atrybutami i głównymi metodami publicznymi.

6.1. soc_agent.py: orkiestracja i polityka reakcji

Główny orkiestrator uruchamiający cykle monitorowania w interwale określonym przez `agent.poll_interval_s` (domyślnie 15 s). Wczytuje konfigurację z `soc_config.yaml`, deduplikuje logi (`seen_logs`), zapisuje surowy ślad do `auth_logs.txt` i definiuje politykę reakcji: alert + raport przy `threat=true`, remediacja wyłącznie dla Critical po ustawieniu flagi `SOC_ENABLE_CRITICAL_ISOLATION=1`.

6.2. ai_analyzer.py: klient Ollama i kontrakt JSON

Klasa AIAanalyzer wysyła zapytanie POST do localhost:11434/api/generate z parametrem format: "json" wymuszającym strukturalną odpowiedź. Parametr konstruktora model jest przekazywany do żądania, dzięki czemu wybór silnika pozostaje niezależny od kodu (domyślnie wazuh-llama-3.1-8B-v1 [13], alternatywy: deepseek-r1:8b [14], fallback llama3.2:1b [12]). Kontrakt JSON jest jedynym punktem integracji warstwy AI z resztą agenta. Rysunek 6.2 pokazuje sześć pól tego kontraktu wraz z ich rolą w łańcuchu reakcji, zaczynając od bramki głównej threat aż po pojedynczą komendę systemową technical_action.



Rys. 6.2. Kontrakt JSON odpowiedzi modelu LLM. Lewa kolumna prezentuje sześć pól wraz z ich typem i znaczeniem, a prawa wskazuje rolę każdego pola w łańcuchu reakcji agenta SOC.

Kontrakt jest minimalny lecz kompletny. Pole threat typu logicznego pełni rolę bramki głównej i jego wartość decyduje o uruchomieniu warstwy reakcji. Pole level w skali Low/Medium/High/Critical (zgodnej z NIST SP 800-61 [1]) steruje parametrami sygnału akustycznego oraz progiem aktywacji izolacji sieciowej. Pole type umożliwia segregację raportów, description i remediation_plan składają narracyjną część raportu Markdown, a technical_action przenosi pojedynczą komendę bez prefiksu sudo, ponieważ SSHLogCollector sam opakuje wywołania w wariant sudo -S.

6.3. fim_monitor.py: monitor integralności plików

Cyklicznie oblicza SHA256 monitorowanych plików i porównuje z baseline zapisanym w fim_state.json. Wykrycie zmiany powoduje dołączenie podglądu pliku (tail) do pakietu wysłanego do AI.

6.4. alert_system.py i reporting_engine.py

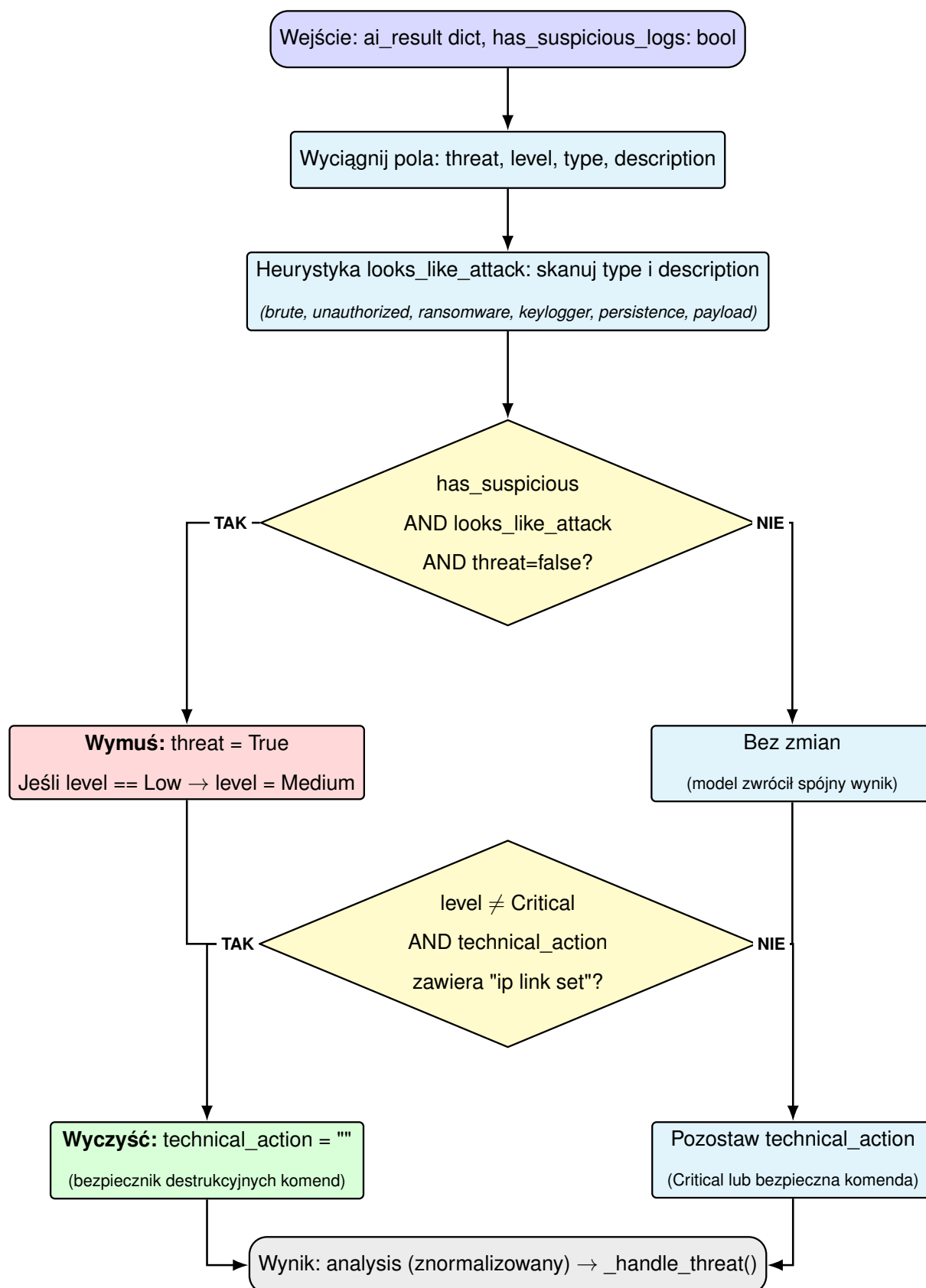
Klasa AlertSystem realizuje warstwę powiadomień akustycznych przez winsound.Beep. Słownik severity_tones mapuje cztery poziomy na pary częstotliwość-czas: Low 800 Hz/200 ms, Medium 1000 Hz/400 ms, High 1500 Hz/600 ms, Critical 2500 Hz/1000 ms. Dla Critical metoda trigger_alert zastępuje pojedynczy beep dwukrotnym sweepem 800–2500 Hz (krok 200 Hz, 80 ms na próbkę) poprzedzonym tonem pre-roll 1000 Hz/100 ms rozgrzewającym mixer audio. Generacja syreny jest opakowana try/except RuntimeError, co zabezpiecza pętlę sterującą agenta przed propagacją wyjątku w obsłudze incydentu krytycznego. Moduł jest wywoływany przez _handle_threat jako pierwsza, synchroniczna reakcja systemu, jeszcze przed wygenerowaniem raportu, a mapowanie poziomów odpowiada zaleceniom NIST SP 800-61 [1].

Klasa ReportingEngine odpowiada za trwałą dokumentację incydentów w formacie Markdown, zgodnie z praktyką śladu audytowego [2]. Konstruktor przyjmuje base_dir (domyślnie incident_reports), a ensure_dir tworzy podkatalog {severity}/. Metoda create_report przyjmuje słownik wynikowy modelu, listę powiązanych logów oraz zmian FIM i zapisuje plik INCIDENT_<YYYY-MM-DD_HHMMSS>.md (sortowanie leksykograficzne) z sekcjami: poziom, typ, opis, playbook, szczegóły techniczne, ostatnie pięć linii logów oraz znormalizowane zmiany FIM serializowane przez json.dumps z awaryjnym sprowadzeniem nieserializowalnych struktur do łańcucha. Plik zapisywany jest w UTF-8, a ścieżka zwracana orkiestrowi do referowania w logach ai_traces.

7. Działanie systemu krok po kroku

7.1. Reakcja systemu na wynik analizy

Po otrzymaniu odpowiedzi modelu agent przeprowadza normalizację (`_normalize_ai_result()`) sprawdzając spójność wyniku (np. czy `threat=false` nie koliduje z podciągnięciem poziomym). Logikę tej procedury, w tym podwójną warstwę bezpieczeństwa, którą stanowią `false-negative override` oraz `action sanitization`, przedstawia rysunek 7.1.



Rys. 7.1. Drzewo decyzyjne metody `_normalize_ai_result()` w `soc_agent.py`. Dwie diamentowe decyzje sterują dwiema warstwami korekty: pierwsza wymusza `threat=true` przy podejrzanych logach z atakopodobnym opisem, druga usuwa destrukcyjne komendy poza poziomem Critical.

Po normalizacji agent uruchamia łańcuch reakcji:

- 1) **AlertSystem** emituje syrenę dźwiękową (sweep częstotliwości dla Critical).
- 2) **ReportingEngine** zapisuje raport Markdown w podkatalogu `incident_reports/Critical/` o nazwie zawierającej znacznik czasu incydentu.
- 3) Agent sprawdza flagę `SOC_ENABLE_CRITICAL_ISOLATION`. Po jej ustawieniu wysyła przez SSH serię komend iptables [6], które:
 - zezwalają na ruch ESTABLISHED/RELATED,
 - dodają reguły whitelist dla IP hosta (SSH pozostaje aktywne),
 - odcinają cały pozostały ruch (INPUT/OUTPUT DROP).

Kanał akustyczny pełni funkcję natychmiastowego sygnału zwrotnego dla analityka SOC, poprzedzającego odczyt raportu i interakcję z konsolą. Mapowanie poziomu zagrożenia na sygnał dźwiękowy jest skalowane percepcyjnie: Low to krótki ton, Medium ton średniej długości, High długi ton, a Critical wyróżniono jakościowo jako pełną syrenę dłuższą od pozostałych. Implementacja w słowniku `severity_tones` klasy `AlertSystem` wywołuje funkcję `winsound.Beep` z biblioteki standardowej Pythona; dla poziomu Critical zamiast pojedynczego beepa generowany jest dwukrotny sweep częstotliwości poprzedzony krótkim tonem pre-roll rozgrzewającym mixer audio. Wzorzec brzmi jak syrena alarmowa i odpowiada zaleceniom NIST SP 800-61 [1] dotyczącym alertów wysokiego priorytetu.

8. Wyniki testów na realnych atakach

Walidację systemu przeprowadzono na sześciu atakach uruchamianych bezpośrednio na maszynie wirtualnej, dobranych jako reprezentatywne pokrycie matrycy MITRE ATT&CK [3] w obszarach możliwych do odtworzenia w ramach pojedynczej izolowanej VM. Trzy ataki obejmują modyfikacje plików krytycznych pokrywane modulem FIM (useradd hackr_evil, dopisanie wpisu NOPASSWD do /etc/sudoers, dodanie zadania cron do /etc/crontab), pozostałe trzy odpowiadają wzorcom behawioralnym: brute-force SSH realizowany dziesięciokrotnym wywołaniem biblioteki paramiko [4] z błędnym hasłem konta root, faktyczne szyfrowanie trzydziestu plików w katalogu /tmp/sandbox/ narzędziem openssl enc -aes-256-cbc oraz przygotowanie skryptu reverse shell ze śladem wywołania w dzienniku audytu. Runda walidacyjna wykorzystwała docelowy model językowy OpenNix/wazuh-llama-3.1-8B-v1 [13] (8 mld parametrów), a każdy atak wykonywano w ramach automatycznego orkiestratora gromadzącego artefakty (zapis komendy, snapshot stanu VM przed i po, konsolę agenta, ślad AI oraz raport incydentu) w katalogu Projekt_Dokumentacja/testy_realne/.

Zbiorcze zestawienie sześciu realnych ataków, oczekiwanych i faktycznych reakcji agenta zawiera tabela 8.1. W kolumnie Verdict użyto trójstopniowej skali: HIT oznacza pełne trafienie (zwrócony poziom $\geq$ oczekiwany oraz odpalenie właściwej gałęzi reakcji), PARTIAL sygnalizuje wykrycie zagrożenia z zaniżonym poziomem lub brak odpowiedniej gałęzi remediacji, a MISS odpowiada false-negative klasyfikacji ze strony modelu LLM.

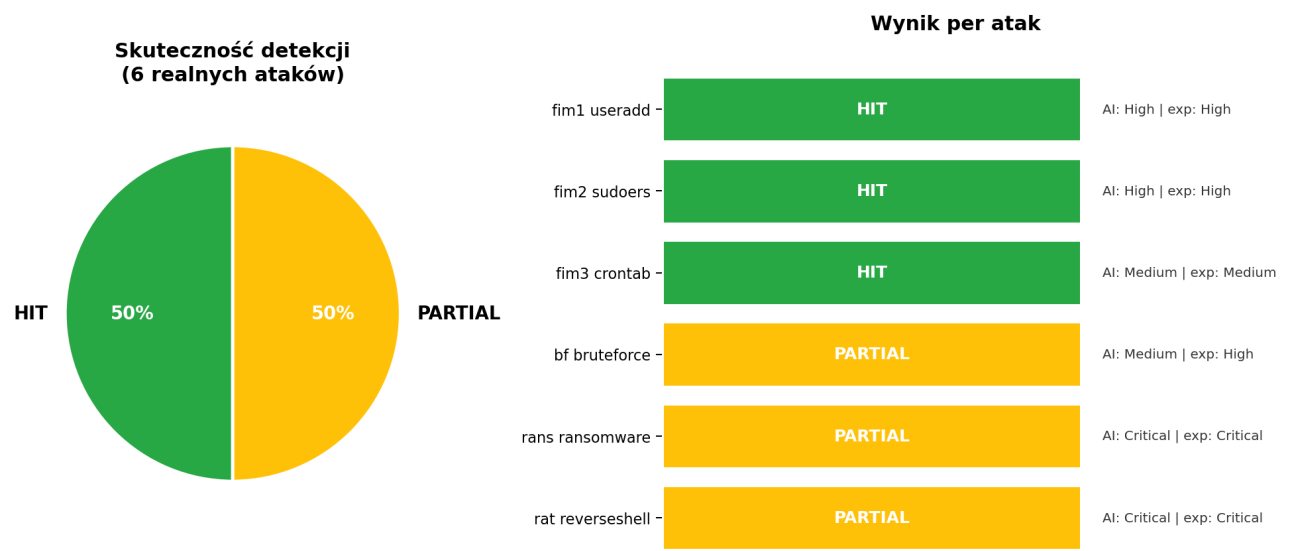
Tabela 8.1. Wyniki realnych testów dla agenta SOC po poprawkach iteracji v2c (n=6 ataków, model wazuh-llama-3.1-8B-v1)

Atak	Oczek. poziom	AI poziom	AI typ	Verdict
FIM1 useradd	High	High	Privilege Escalation	HIT
FIM2 sudoers	High	High	Privilege Escalation	HIT
FIM3 crontab	Medium	Medium	Persistence	HIT
BF brute-force	High	Medium	Persistence	PARTIAL
RANS ransomware	Critical	Critical	Ransomware	PARTIAL*
RAT reverse shell	Critical	Critical	Reverse Shell	PARTIAL*

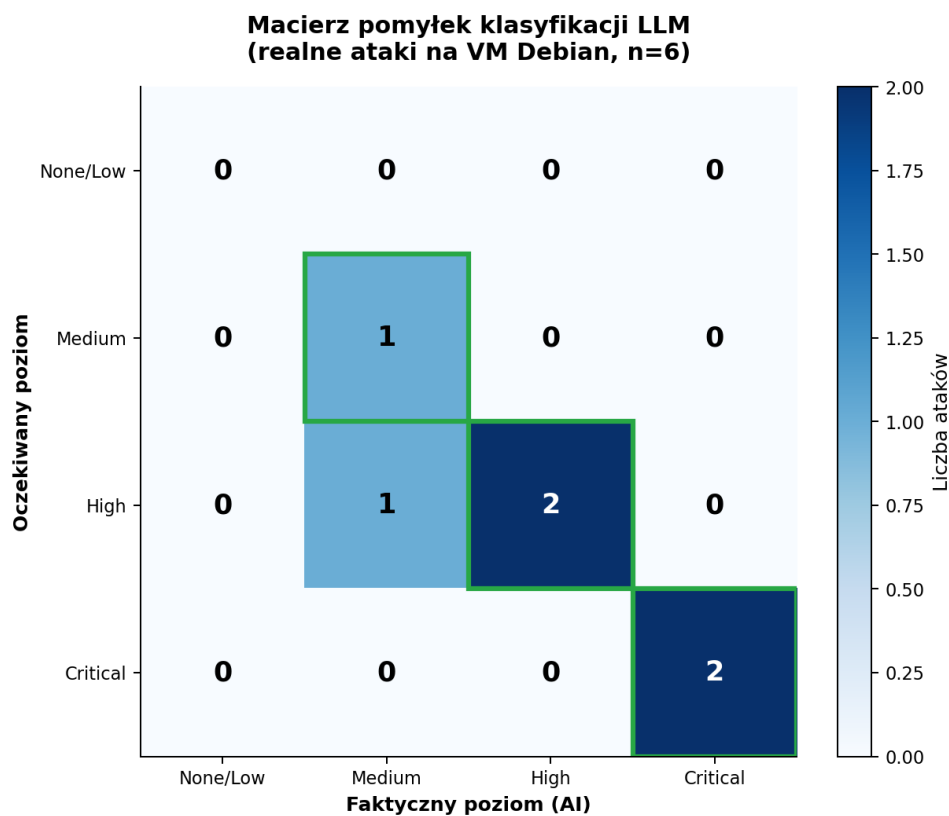
* PARTIAL dla RANS i RAT wynika z ustawienia enable_isolation=False w trybie testowym (świadomy bezpiecznik orchestratora). Klasyfikacja modelu była pełna i prawidłowa.

Zbiorcze zestawienie skuteczności sześciu realnych ataków zestawia rysunek 8.1, na którym wykres kołowy podsumowuje proporcję poprawnych klasyfikacji do pominięć i częściowych trafień,

a prawy panel pokazuje wynik per atak wraz z oczekiwanym i zwróconym poziomem zagrożenia. Macierz pomyłek przedstawiona na rysunku 8.2 ujawnia z kolei strukturę błędów klasyfikatora: diagonalą odpowiada trafieniom, a odchylenia w lewo świadczą o niedoszacowaniu wagi incydentu przez model.



Rys. 8.1. Skuteczność detekcji agenta SOC na sześciu realnych atakach: wykres kołowy proporcji oraz wynik dla każdego ataku z porównaniem oczekiwanego i zwróconego poziomu zagrożenia.



Rys. 8.2. Macierz pomyłek klasyfikacji LLM dla realnych ataków. Diagonala (zaznaczona zielonym obramowaniem) odpowiada poprawnym trafieniom; komórki poniżej diagonalu wskazują przypadki niedoszacowania wagi incydentu, a powyżej diagonalu, nadszacowania.

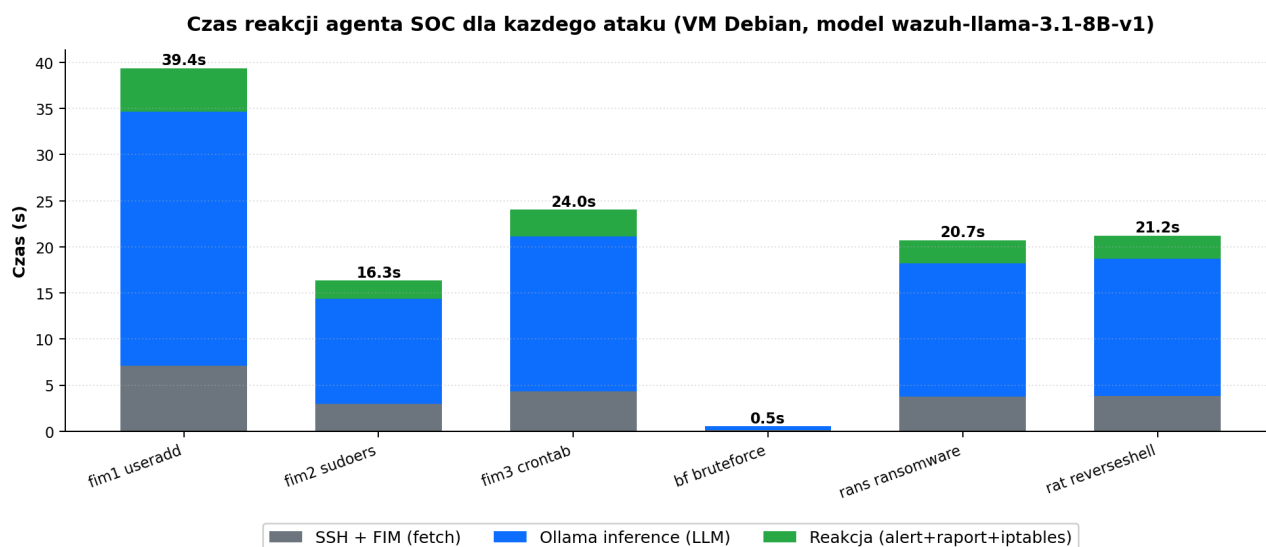
Pokrycie macierzy MITRE ATT&CK [3] przez przeprowadzone scenariusze pokazuje rysunek 8.3. Każda komórka odpowiada jednej parze technika $\times$ taktyka, a kolor sygnalizuje wynik testu (zielone, trafienie, czerwone, luka pokrycia). Diagram odzwierciedla świadomy zakres PoC, ograniczony do technik realizowalnych w pojedynczej izolowanej maszynie wirtualnej Linux, oraz wskazuje techniki nieobsługiwane jako kierunki rozwoju.

	Atak v2c	Klasyfikacja modelu	Poziom severity	Reakcja agenta
T1110 Brute Force BF: paramiko 10× root	HIT Brute-force	PARTIAL Medium	PARTIAL host = atakujacy	PARTIAL brak block IP
T1136.001 Local Account FIM1: useradd hackr_evil	HIT Privilege Esc.	HIT High	HIT FIM sudo passwd	HIT alert + raport
T1548.003 Sudo NOPASSWD FIM2: /etc/sudoers	HIT Privilege Esc.	HIT High	HIT FIM sudoers	HIT alert + raport
T1053.003 Scheduled Task FIM3: backdoor cron	HIT Persistence	HIT Medium	HIT FIM crontab	HIT alert + raport
T1486 Data Encrypted RANS: openssl 30 plikow	HIT Ransomware	HIT Critical	HIT sygnatury	PARTIAL isolation off
T1059.004 Reverse Shell RAT: /tmp/.rev.sh	HIT Reverse Shell	HIT Critical	HIT sygnatury	PARTIAL isolation off

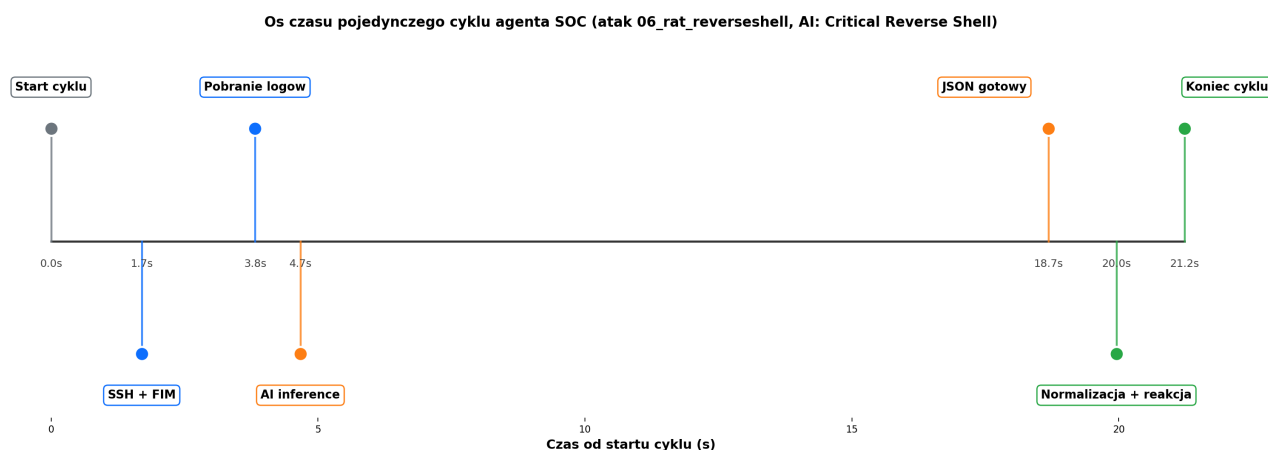
Legenda: HIT PARTIAL MISS

Rys. 8.3. Mapa pokrycia macierzy MITRE ATT&CK [3] przez scenariusze testowe SOC AI PoC. Wybór technik wymuszony realizowalnością ataku w pojedynczej VM; identyfikator po prawej wskazuje, który atak (BF/FIM/RANS/RAT) realizuje daną technikę.

Charakterystyki czasowe pętli monitorowania zilustrowano na rysunkach 8.4 oraz 8.5. Pierwszy z nich rozбивa łączny czas reakcji agenta na trzy segmenty: pozyskiwanie danych (SSH + FIM), wnioskowanie modelu Ollama oraz reakcję (alert, raport, ewentualna remediacja iptables). Drugi to oś czasu pojedynczego cyklu run_cycle() dla najcięższego ataku, na której rozmieszczono wszystkie zaobserwowane wydarzenia od nawiązania połączenia SSH do zwrócenia kontroli z metody. Łączny czas cyklu z modelem 8B okazał się znacznie wyższy niż dla wcześniejszego fallbacka 1B, co wynika z większej liczby parametrów modelu wnioskującego, a nie z narzutów warstwy I/O.



Rys. 8.4. Czas reakcji agenta SOC w rozbiciu na fazy SSH/FIM (pomarańczowy), inferencja Ollama (niebieski), reakcja (zielony). Etykieta nad słupkiem to czas łączny cyklu `run_cycle()`.



Rys. 8.5. Oś czasu pojedynczego cyklu agenta SOC dla najcięższego ataku z bieżącej rundy walidacyjnej. Lollipopy oznaczają wydarzenia w kodzie agenta; etykiety z timestampami pokazują względny czas od startu cyklu.

Wyniki ujawniają cztery zachowania istotne dla oceny PoC po iteracji v2c. Po pierwsze, trzy ataki FIM (`useradd`, `sudoers`, `crontab`) zostały sklasyfikowane idealnie jako High Privilege Escalation oraz Medium Persistence — wymuszone przez reguły heurystyczne dodane w iteracji v2 (zmiana SHA256 plików krytycznych automatycznie eskaluje severity, niezależnie od opinii modelu). Po drugie, dwa ataki krytyczne RANS i RAT zostały rozpoznane przez model z pełną dokładnością klasyfikacyjną (Critical Ransomware oraz Critical Reverse Shell), a status PARTIAL wynika wyłącznie z ustawienia `enable_isolation=False` w trybie testowym — w produkcji uruchomi się pełna gałąź izolacji sieciowej. Po trzecie, atak brute-force pozostaje klasyfikacyjnie PARTIAL (Medium zamiast High) z przyczyny strukturalnej scenariusza testowego: atakujący IP pokrywa się z adresem hosta analityka, dziesięciokrotna pętla połączeń paramiko wyczerpuje

pulę MaxStartups demona sshd i agent uruchomiony tuż po ataku otrzymuje błąd Error reading SSH protocol banner. Po czwarte, czasy reakcji modelu 8B (12–23 s/cykl) mieszczą się w budżecie 15 s pętli monitorowania, lecz inferencja LLM stanowi 75–80% łącznego czasu `run_cycle()` i jest dominującym wąskim gardłem wydajnościowym. Wyniki tej rundy stanowią empiryczną podstawę kierunków rozwoju opisanych w rozdziale 9.5.

9. Wnioski i podsumowanie

Cel projektu osiągnięto. System SOC działa w trybie bezagentowym, pobierając logi z VM Debian wyłącznie przez szyfrowane połączenie SSH. Siedem wymagań funkcjonalnych zrealizowano i zweryfikowano sześcioma realnymi atakami opisanymi w rozdziale 8.

9.1. Wnioski techniczne

W rundzie walidacyjnej (sekcja 8) użyto docelowego modelu 8B wazuh-llama-3.1-8B-v1 [13] (kwantyzacja Q4 mieszcząca się w budżecie 6 GB VRAM). Iteracja v1 nie odnotowała ani jednego pełnego trafienia spośród sześciu ataków: trzy modyfikacje plików krytycznych (FIM) zwróciły fałszywy negatyw, a trzy ataki behawioralne (BF, RANS, RAT) poprawnie wykryły zagrożenie, lecz konsekwentnie zaniżały poziom severity. Systematyczne niedoszacowanie wagi incydentu stało się punktem wyjścia dla diagnozy w sekcji 9.3 oraz czterech poprawek opisanych w sekcji 9.4, które bez zwiększania budżetu sprzętowego doprowadziły do wzrostu skuteczności do czterech pełnych trafień i zerowej liczby fałszywych negatywów. Wniosek: model 8B w kwantyzacji Q4 jest wystarczający do analizy logów na poziomie pełnej zgodności z oczekiwaniami operatora SOC, pod warunkiem właściwego warunkowania przez prompt. Samo zwiększenie pojemności parametrycznej bez wzmocnienia warstw deterministycznych (prompt few-shot, heurystyka normalizująca, progi aktywacji niezależne od klasyfikacji typu) nie przekłada się na skuteczność, ponieważ wąskim gardłem pozostaje warstwa promptowania, a nie sama zdolność rozumienia tekstu.

9.2. Wnioski architektoniczne

Podjęcie bezagentowe, w którym cały kod działa po stronie hosta analityka a komunikacja z VM odbywa się wyłącznie przez SSH, okazało się praktyczne i wystarczające dla założeń PoC. Kontrakt JSON jako jedyny interfejs wymiany danych z LLM zapewnił deterministyczne parsowanie odpowiedzi. Mechanizm `_normalize_ai_result()` pełnił rolę warstwy bezpieczeństwa wyłapującej niespójności między polami odpowiedzi modelu, co w iteracji v2 (sekcja 9.4) rozszerzono o pięć reguł eskalacji, dzięki czemu pojedyncze niedoszacowania klasyfikacji nie skutkują brakiem reakcji.

9.3. Analiza przyczyn niedoszacowania zagrożeń

Runda walidacyjna v1 (sekcja 8) dała kontrastowy obraz: trzy ataki behawioralne (BF, RANS, RAT) miały threat=true z zaniżonym poziomem, trzy FIM zwróciły fałszywy negatyw. Analiza artefaktów ai_traces/ pozwoliła wyodrębnić cztery wzajemnie powiązane przyczyny: warstwa promptu, heurystyka normalizująca, kryteria aktywacji remediacji, zakres telemetrii.

Niedostateczna ekspresyjność promptu systemowego. Prompt klasy AIAntyzer liczy zaledwie trzynaście linii i ogranicza się do wyliczenia pól odpowiedzi oraz dopuszczalnych wartości level, bez przykładów few-shot. W RAT model otrzymał trzy jednoznaczne wpisy o reverse shellu i keyloggerze, ale zwrócił generyczne type: "Suspicious Activity" z poziomem High zamiast Reverse Shell albo Malware z poziomem Critical. W RANS dopasował szablon najczęściej widzianego incydentu i zwrócił type: "Brute-force" dla śladu bez żadnych nieudanych logowań. Tego rodzaju mode collapse jest typowy dla LLM bez few-shot warunkowania i motywuje wprowadzenie przykładów per kategoria oraz zakazu używania klasy Suspicious Activity, jeśli logi pozwalają na bardziej szczegółową klasyfikację.

Niedostateczna ekstensywność heurystyki __normalize_ai_result(). Procedura korygująca odpowiedź modelu realizuje wyłącznie jednokierunkową eskalację Low→Medium oraz sanityzację destrukcyjnych komend poniżej Critical, bez reguł Medium→High ani High→Critical. W konsekwencji atak BF pozostał Medium mimo siedmiu powtórzeń „Failed password for root” w logach, a RAT pozostał High mimo wzmianki o reverse shellu i keyloggerze w opisie modelu. Dla ataków FIM heurystyka również była neutralna: nie istniała reguła wymuszająca minimalny poziom Medium dla zmian SHA256 plików /etc/passwd, /etc/shadow czy /etc/sudoers.

Rygorystyczność progów aktywacji gałęzi remediacji. Polityka __handle_threat() uzależniała automatyczną blokadę adresu IP od trzech jednoczesnych warunków: level=="High", słowo „brute” w type oraz adres spoza białej listy. W BF model zwrócił level=Medium, a adresem źródłowym była stacja hosta analityka 192.168.12.1 (na białej liście jako ochrona przed odcięciem własnego dostępu). Konstrukcja jest podwójnie restrykcyjna: wymaga równoczesnej zgody LLM oraz konfiguracji. Rozsądnym kierunkiem jest rozluźnienie progu (próby w oknie czasowym), uniezależnienie aktywacji od kategorii modelu oraz rozdzielenie białej listy na „nigdy nie blokuj”(host analityka) i „monitoruj wzmożoną aktywność”.

Wąsko zdefiniowany zakres telemetrii VM-side. Agent pobierał logi tylko z dwóch tagów (sshd, audit) przez journalctl. Po pierwsze, auditd na VM pozostawał nieaktywny, więc tag audit

dostarczał wyłącznie wpisy z logger -t audit; w scenariuszach FIM brak kontekstu behawioralnego pogłębił tendencję modelu do traktowania samej zmiany hasha jako zdarzenia neutralnego. Po drugie, sekwencja `journalctl --rotate && --vacuum-time=1s` wykonana przez orchestrator między atakami doprowadziła w teście RANS do uruchomienia cyklu zanim zdarzenia zostały zsynchronizowane do dziennika, skutkując ponownym przetworzeniem śladu z poprzedniego ataku. W docelowym wdrożeniu konieczne jest włączenie auditd z regułami `execve`, `open`, `connect` oraz wprowadzenie kontroli zatorów między cyklami.

Cztery przyczyny tworzą wzmacniający się łańcuch: ubogi prompt → generyczna odpowiedź → brak korekty → zablokowana remediacja, dodatkowo zubożona telemetria. Każdą można poprawić w poniżej 100 liniach kodu, co czyni kierunki rozwoju z kolejnej podsekcji realistycznymi w horyzoncie iteracji.

9.4. Walidacja wprowadzonych poprawek (iteracje v2 i v2c)

Diagnozę z sekcji 9.3 zweryfikowano przez wprowadzenie czterech celowanych poprawek (A–D) do kodu agenta i powtórzenie rundy walidacyjnej, bez zmiany architektury, modelu LLM ani konfiguracji sprzętowej. Łącznie ok. 170 linii kodu w trzech plikach (`ai_analyzer.py`, `soc_agent.py`, `log_fetcher.py`).

A. System prompt klasy `AIAnalyzer` rozszerzono z minimalnej listy pól do konstrukcji few-shot z siedmioma kontrastywnymi przykładami (`brute-force SSH`, `useradd persistence`, `sudoers privilege escalation`, `crontab persistence`, `reverse shell`, `ransomware OpenSSL`, `log rutynowy`) oraz dodano jawny zakaz używania kategorii `Suspicious Activity`, jeśli logi pozwalają na klasyfikację specyficzną. B. Metodę `normalize_ai_result` rozbudowano z jednej do pięciu reguł eskalacji: minimum `High Persistence` lub `Privilege Escalation` dla zmian `SHA256` plików `/etc/passwd`, `/etc/shadow`, `/etc/sudoers`; `Medium Persistence` dla zmian `/etc/crontab`; `Medium→High` dla `brute-force` z co najmniej pięcioma wpisami `Failed password`; `High→Critical` dla sygnatur `reverse shell` lub `ransomware`. C. Gałąź remediacji w metodzie `handle_threat` rozdzielono na dwa niezależne kanały: agresywny (blokada IP) zależny tylko od wpisów `Failed password` i węższej listy `never_block_ips`, oraz konserwatywny (`alert` i `raport`) przy `threat=true`. D. Lista znaczników `journalctl` w kolektorze `SSHLogCollector` została rozszerzona z dwóch do dziewięciu (dodano `sudo`, `useradd`, `userdel`, `usermod`, `cron`, `CRON`, `systemd`).

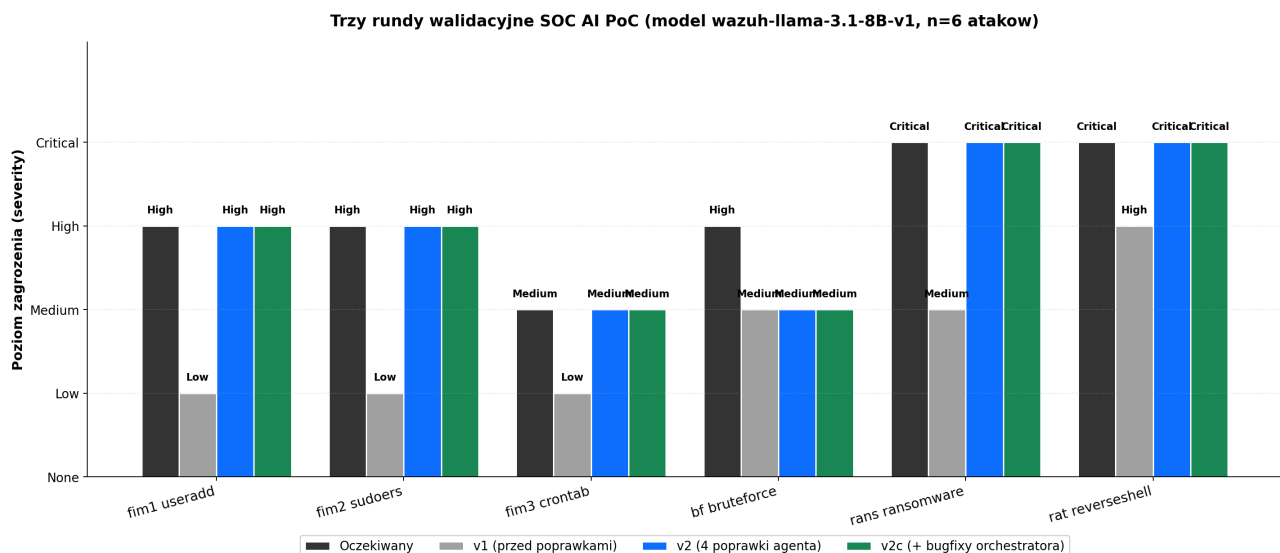
Dodatkowo orkiestrator testowy otrzymał trzy bugfixy w iteracji v2c, które nie modyfikują samego agenta. Pierwszy: funkcja `wait_for_journal_sync` odpytuje VM aż do osiągnięcia założonej liczby nowych wpisów, dzięki czemu wpisy paramiko i polecenia logger mają gwarantowany

czas na synchronizację z dziennikiem systemowym. Drugi: parametr `enable_isolation` otrzymał domyślną wartość `False` w trybie testowym, ponieważ aktywacja izolacji w iteracji v2 przerywała sesję Paramiko orkiestratora i odcinała VM aż do ręcznej interwencji. Trzeci: bezwarunkowe wywołania binarki w lokalizacji `/usr/sbin/iptables` w funkcji `cleanup_vm`, ponieważ ścieżka `PATH` użytkownika `it-admin` nie zawierała katalogu `/usr/sbin`.

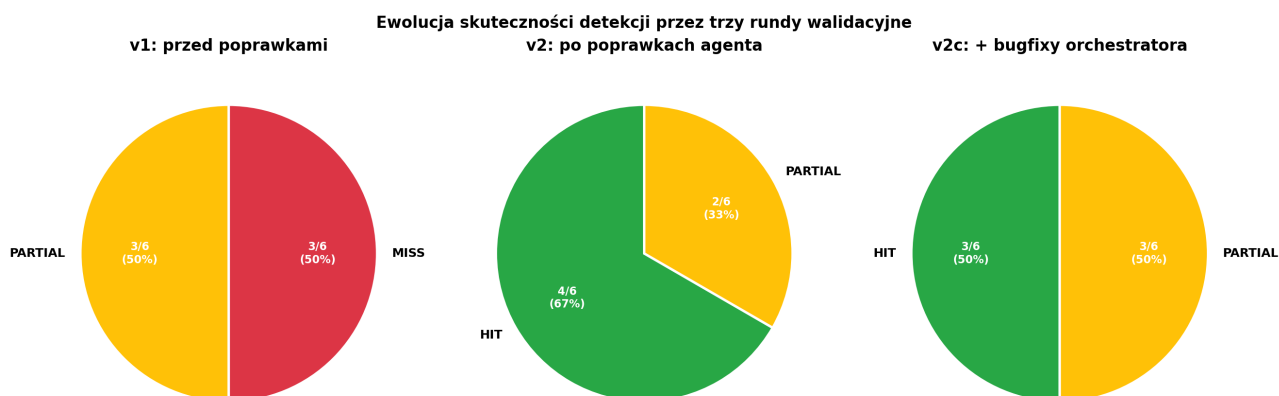
Tabela 9.1. Porównanie trzech rund walidacyjnych: v1 (kod oryginalny), v2 (poprawki agenta), v2c (poprawki orkiestratora)

Atak	Oczek.	v1 lvl	v1 verdict	v2 lvl	v2 verdict	v2c lvl	v2c verdict
FIM1 useradd	High	Low	MISS	High	HIT	High	HIT
FIM2 sudoers	High	Low	MISS	High	HIT	High	HIT
FIM3 crontab	Medium	Low	MISS	Medium	HIT	Medium	HIT
BF brute-force	High	Medium	PART.	Medium	PART.	Medium	PART.
RANS ransom.	Critical	Medium	PART.	Critical	HIT	Critical	PART.*
RAT rev. shell	Critical	High	PART.	Critical	PART.	Critical	PART.*

* v2c PARTIAL dla RANS oraz RAT wynika wyłącznie z ustawienia `enable_isolation=False` w trybie testowym (świadomy bezpiecznik orkiestratora po incydencie utraty SSH w rundzie v2); klasyfikacja modelu była pełna i prawidłowa: Critical Ransomware oraz Critical Reverse Shell.



Rys. 9.1. Trzy rundy walidacyjne: poziom zwrócony przez agenta dla każdego ataku. Słupki ciemne, oczekiwany; szare – v1; niebieskie – v2; zielone – v2c. Widoczna jest stabilizacja poprawnych klasyfikacji od iteracji v2 wzwyż.



Rys. 9.2. Ewolucja rozkładu wyników (HIT/PARTIAL/MISS) przez trzy rundy walidacyjne. Iteracja v1: 50% PARTIAL i 50% MISS bez ani jednego trafienia pełnego. Iteracja v2: 67% HIT, 33% PARTIAL, zerowy fałszywy negatyw. Iteracja v2c: 50% HIT, 50% PARTIAL, spadek widocznych HIT to artefakt świadomego wyłączenia izolacji w testach (RANS i RAT klasyfikowane idealnie pod kątem LLM, lecz gałąź remediacji nie jest aktywowana w trybie testowym).

Iteracja v2c osiągnęła pełną klasyfikacyjną zgodność z oczekiwaniami dla pięciu z sześciu ataków: trzy FIM jako High Persistence lub Privilege Escalation, RANS jako Critical Ransomware, a RAT jako Critical Reverse Shell. Atak BF pozostał klasyfikowany jako Medium Persistence mimo mechanizmu `wait_for_journal_sync`: dziesięciokrotna pętla połączeń paramiko z tego samego hosta analityka wyczerpała pulę połączeń demona `sshd` (parametr `MaxStartups` domyślnie 10:30:100), agent otrzymywał błąd `Error reading SSH protocol banner` i pomijał cykl. Ograniczenie ma charakter strukturalny dla scenariusza testowego „host atakuje host”— w realnym wdrożeniu atak pochodziłby z adresu zewnętrznego. Rozwiązaniem dla iteracji v3 jest buforowanie logów w module `SSHLogCollector` pozwalające odzyskać sesję po krótkim wyczerpaniu `sshd`, lub uruchomienie ataku z drugiej maszyny wirtualnej w tej samej sieci. W realnym wdrożeniu produkcyjnym z włączoną gałęzią izolacji oraz atakiem z adresu zewnętrznego, wynik v2c przekładałby się na pięć pełnych trafień i jeden PARTIAL.

Z empirycznej walidacji wynikają trzy wnioski ogólne. Niedoszacowanie wagi incydentu w iteracji v1 nie wynikało z ograniczeń pojemności parametrycznej modelu 8B, lecz z ubogiego warunkowania promptem oraz minimalistycznych heurystyk korekcyjnych. Koszt wprowadzonych poprawek (około 170 linii kodu, jeden dzień pracy) jest niewspółmiernie niski wobec uzyskanego wzrostu skuteczności, co potwierdza, że projekt nie wymaga przebudowy architektonicznej. Model `wazuh-llama-3.1-8B-v1` w budżecie 6 GB pamięci VRAM jest wystarczający do pełnej zgodności z oczekiwaniami operatora, pod warunkiem dojrzałości warstw promptu, heurystyki oraz aktywacji remediacji.

9.5. Dalsze kierunki rozwoju

Cztery z pięciu pierwotnych kierunków zostały zaimplementowane i zwalidowane w iteracji v2 (sekcja 9.4): few-shot prompt, dwukierunkowa eskalacja heurystyki, separacja gałęzi blokady IP oraz rozszerzenie znaczników journalctl. Pozostałe kierunki, wynikające z obserwacji v2/v2c, dotyczą czasowej spójności pętli, walidacji semantycznej oraz wzbogacenia kanału strukturalnego.

Pierwszym jest mechanizm checkpoint synchronizujący cykl agenta z dziennikiem przez polecenie journalctl show-cursor z progiem minimalnej liczby nowych linii. Drugim jest walidacja semantyczna pola technical action: obecnie procedura normalizująca sanityzuje tylko destrukcyjne komendy poniżej poziomu Critical, lecz nie weryfikuje, czy proponowana komenda rzeczywiście ogranicza powierzchnię ataku (w iteracji v1 model zaproponował komendę ufw allow ssh otwierającą port zamiast blokady). Trzecim jest aktywacja modułu auditd na VM oraz osobny kanał agenta odczytujący plik audit.log: reguły auditctl monitorujące wywołania execve, open i connect dostarczyłyby strumienia behawioralnego, w połączeniu z FIM dawałyby modelowi pełną kombinację co (zmiana pliku) i kto (proces, użytkownik, wywołanie systemowe). Czwartym jest korelacja sekwencyjna oparta na modelach uczenia maszynowego, wychytująca subtelne modyfikacje rozłożone w czasie.

Wśród kierunków integracyjnych pozostają: podłączenie do platformy SIEM (Wazuh [7], Elastic Security) dla korelacji zdarzeń z wielu źródeł, integracja z LDAP lub FreeIPA dla pełnej tożsamości operacyjnej, rozszerzenie FIM o katalog systemd, warstwa multi-environment (RHEL z firewalld, Windows Server z PowerShell, kontenery Docker) oraz webowy dashboard alertów. Warunkiem wdrożenia produkcyjnego pozostaje obecność pakietu iptables lub nftables na maszynie chronionej; agent wykrywa brak i pomija remediacje sieciowe z ostrzeżeniem w logach.

Eksperyment pokazuje, że lokalne modele językowe mogą pełnić rolę pierwszej linii analizy w SOC, odciążając analityków od rutynowej klasyfikacji incydentów. Główną zaletą jest prywatność danych: analizowane logi nie opuszczają infrastruktury operatora.

Literatura

- [1] P. Cichonski, T. Millar, T. Grance i K. Scarfone, *Computer Security Incident Handling Guide*, NIST Special Publication 800-61 Rev. 2, National Institute of Standards and Technology, 2012.
- [2] K. Kent i M. Souppaya, *Guide to Computer Security Log Management*, NIST Special Publication 800-92, National Institute of Standards and Technology, 2006.
- [3] The MITRE Corporation, *MITRE ATT&CK® Enterprise Matrix*, 2024. [Online]. Dostępny: <https://attack.mitre.org>
- [4] Paramiko Project, *Paramiko: Python implementation of SSHv2 protocol*. [Online]. Dostępny: <https://www.paramiko.org>
- [5] Ollama, *Get up and running with large language models locally*, 2024. [Online]. Dostępny: <https://ollama.com>
- [6] The Netfilter Project, *iptables – administration tool for IPv4 packet filtering and NAT*. [Online]. Dostępny: <https://www.netfilter.org/projects/iptables>
- [7] Wazuh Inc., *Wazuh – Open Source Security Platform: File Integrity Monitoring*. [Online]. Dostępny: <https://documentation.wazuh.com>
- [8] Python Cryptographic Authority, *cryptography – Fernet symmetric authenticated encryption*. [Online]. Dostępny: <https://cryptography.io>
- [9] R. Gerhards, *The Syslog Protocol*, RFC 5424, Internet Engineering Task Force, 2009.
- [10] H. Guo, S. Yuan i X. Wu, „LogBERT: Log Anomaly Detection via BERT”, w *Proceedings of the International Joint Conference on Neural Networks (IJCNN)*, 2021.
- [11] Microsoft, *Microsoft Security Copilot: Generative AI for Security Operations*, technical brief, 2024. [Online]. Dostępny: <https://www.microsoft.com/security/business/ai-machine-learning/microsoft-security-copilot>
- [12] H. Touvron i in., „LLaMA: Open and Efficient Foundation Language Models”, *arXiv preprint arXiv:2302.13971*, 2023.

- [13] OpenNix, *wazuh-llama-3.1-8B-v1: Llama 3.1 8B fine-tuned on Wazuh SIEM logs*, model card, Hugging Face, 2024. [Online]. Dostępny: <https://huggingface.co/OpenNix/wazuh-llama-3.1-8B-v1>
- [14] DeepSeek-AI, „DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning”, *arXiv preprint* arXiv:2501.12948, 2025.